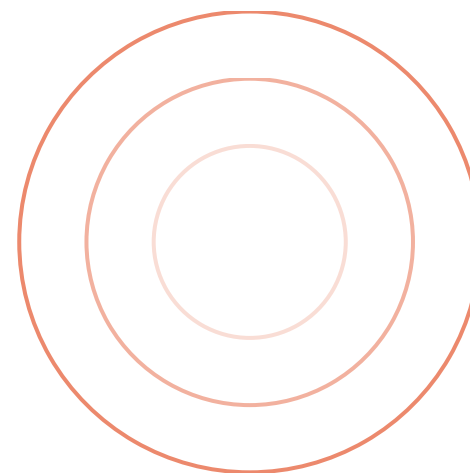


CLOUDLET'S SERVERLESS

From Knative to the Portal

What we built, and why.

Team review · September 2026



TODAY

What we'll talk about

- What is serverless computing
- What is Knative and OpenShift Serverless Operator
- The API: active-active Knative wrapper
- Functions: buildpacks, kpack, build controller
- The portal and cloudlet-apis
- Where we go next

34

slides · six topics · one platform

Serverless, defined

- The servers exist, you stop managing them
- You hand over a unit of code
- The platform decides where it runs
- It runs only while work arrives
- You are billed for that time

**serverless $\neq$ no
servers**

someone else's problem, on purpose

What that buys you

- Scale to zero when idle
- Scale out when traffic arrives
- Pay for requests, not hours
- Nothing to patch, nothing to plan
- One deploy, no capacity meeting

0 → N → 0

replicas follow traffic

Serverless: the trade-offs

	PRO	CON
Cost	Pay only when it runs	Bills spike under load
Ops	No servers to patch	Runtime is not yours
Scale	Up and down by itself	Cold start on first hit
Speed	A function in minutes	Platform lock-in
Fit	Bursty, event-driven	Long-running jobs, state
Deploy	Easy, few clicks	Less control and customization

Knative, and where it came from

- Google open-sourced it in 2018
- A CNCF project since 2022
- Serverless building blocks for Kubernetes
- Two halves: Serving and Eventing
- We run Serving only

2018

Google open-sources Knative

2022

donated to the CNCF

Serving

request-driven workloads

Eventing

brokers, triggers, CloudEvents

Ours

Serving, via the Operator

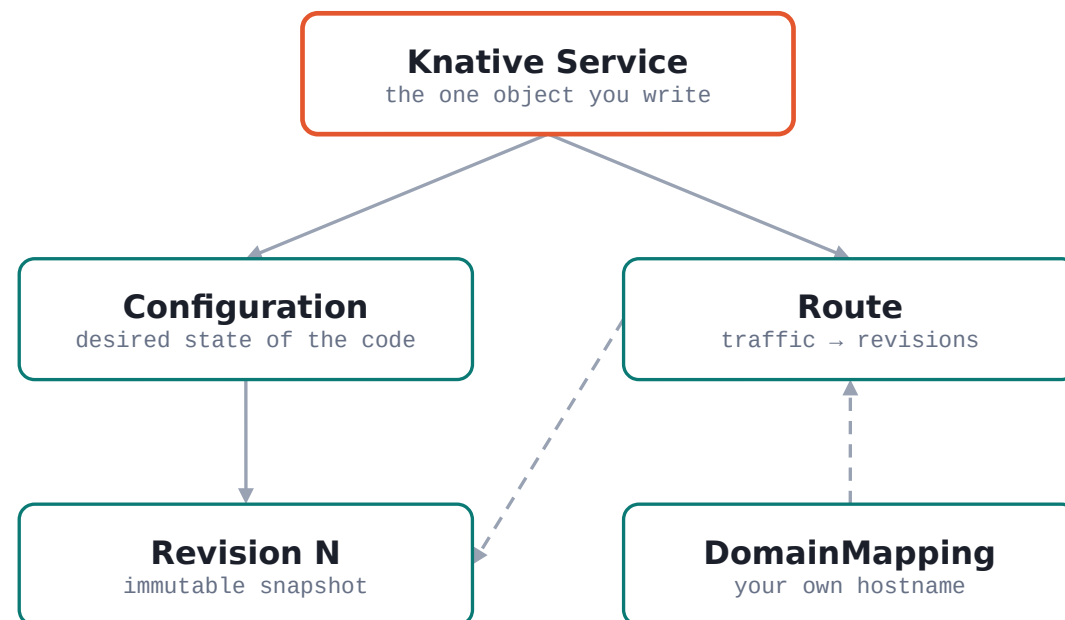
Knative, from what we already run

- We know Deployment, Service, Ingress, HPA
- Knative bundles them into one object
- Adds revisions, routing and scale-to-zero
- Runs on any Kubernetes, ours is OpenShift

NEED	PLAIN KUBERNETES	KNATIVE
Run the code	Deployment	Revision
Reach it	Service + Ingress	Route
Scale it	HPA, never to zero	KPA, down to zero
Change it	Rolling update	New Revision, old kept
Own hostname	Ingress rule	DomainMapping

One object: the Knative Service

- The Service owns Configuration and Route
- Configuration holds the desired code
- Every change is an immutable Revision
- Route sends traffic to revisions
- DomainMapping gives it your hostname



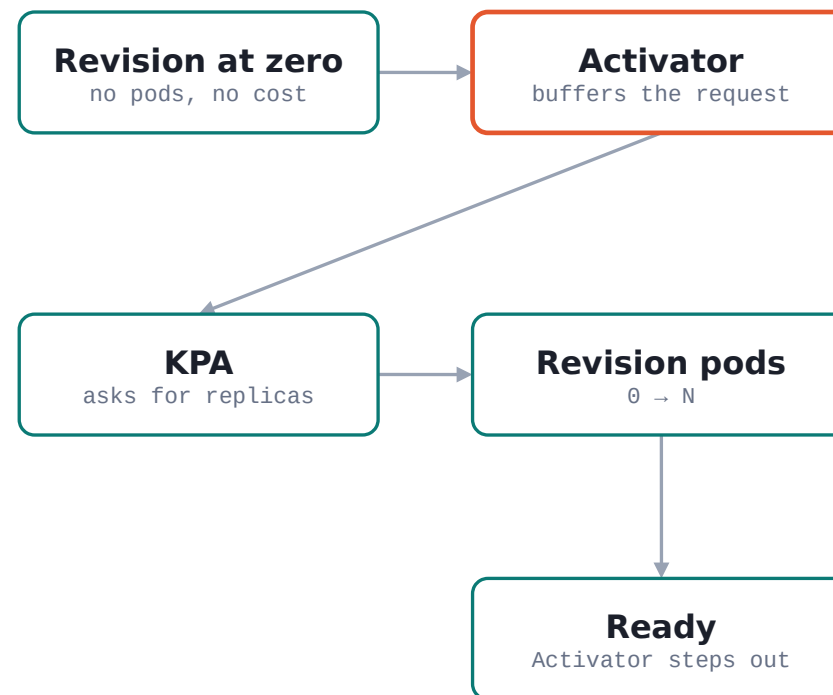
The pieces that do the scaling

- Concurrency: requests in flight per pod
- The queue proxy measures it
- The KPA turns that into replicas
- The Activator stands in at zero
- minScale and maxScale bound it

PIECE	WHAT IT DOES	ROLE
Queue proxy	Counts in-flight requests	the meter
KPA	Requests → replica count	the decision
Activator	Holds requests while pods start	the safety net
minScale	Keeps N pods warm	the override
maxScale	Caps the fan-out	the ceiling

Scaling to zero, and back

- An idle revision drops to zero pods
- The next request reaches the Activator
- The Activator asks the KPA for capacity
- Pods start and the request continues
- Once healthy, traffic goes direct



The OpenShift Serverless Operator

- Red Hat's supported Knative distribution
- One KnativeServing custom resource
- OLM installs it and upgrades it
- Kourier ingress, Routes created for you
- Catalog mirrors with oc-mirror

Knative Serving

Service · Revision · Route ·
DomainMapping

KnativeServing CR

one document

Serverless Operator

reconciles, upgrades

**Operator Lifecycle
Manager**

Subscription → catalog

OpenShift

one cluster per region

Why the Operator

- Every alternative made us own more
- We never create a Route ourselves
- Airgap needs one supported mirror path

	UPSTREAM KNATIVE	SERVERLESS OPERATOR
Upgrades	YAML we maintain	OLM, from the catalog
Ingress	Run it ourselves	Kourier, managed
Routes, TLS	Hand-made, RBAC	Created for us
Airgap	Image by image	oc-mirror, once
Support	Community	Red Hat

The API

One HTTP call, a serverless application.

One call, every cluster

- Functions from git, containers from images
- Validate now, answer 202, poll status
- Closed status vocabulary, published on /info
- FastAPI, Pydantic, Kubernetes-client, cloudlet-apis
- One Helm chart, rendered per region

1

call to deploy everywhere

23

endpoints

3

runtimes

0

kubectl for users

The endpoint structure

- 23 endpoints under one base path
- The group is in the path
- Both offerings have the same shape
- OIDC token or admin key
- Allow live streaming with short-lived tickets

```
/api/serverless/v1
/groups/{group}/functions
  POST · GET · PUT · DELETE
  /{name}/build
  /{name}/stats · /pods · /logs
/groups/{group}/containers
  POST · GET · PUT · DELETE
  /{name}/pull
  /{name}/stats · /pods · /logs
/functions/info · /containers/info
/stream-tickets
```

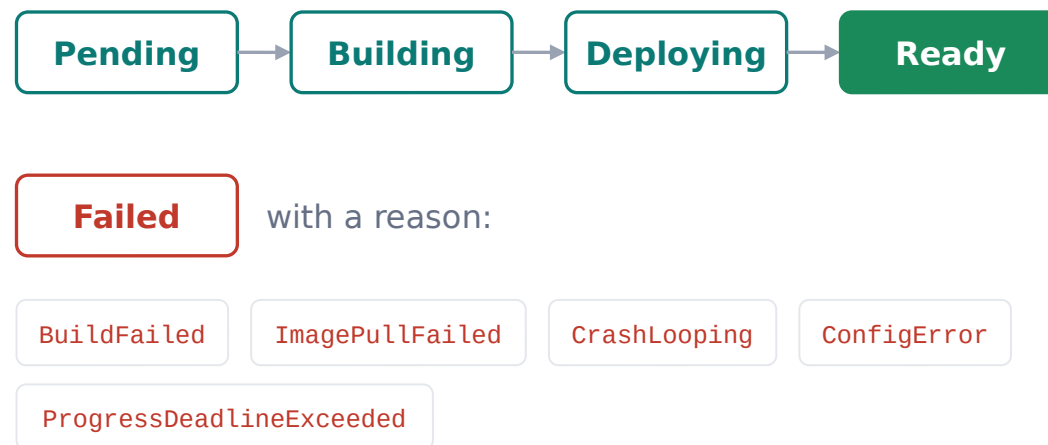
Anatomy of a workload

- Identity: name, hostname, regions
- Source: git repo or image
- Runtime: env, files, port, size
- Scaling: min, max, metric, target
- Response: same shape, secrets redacted

Identity`name · hostname · regions[]`**Source**`gitRepo · branch · runtime | image`**Runtime**`env[] · files[] · port · size`**Scaling**`minScale · maxScale · metric · target`**Response**`the request, secrets redacted`

Every workload has one status

- A closed set, published on /info
- Pending, Building, Deploying, Ready
- Building only exists for functions
- Failed always carries a reason
- One row per region, rolled up



A request, and what comes back

POST /api/serverless/v1/groups/team-a/functions

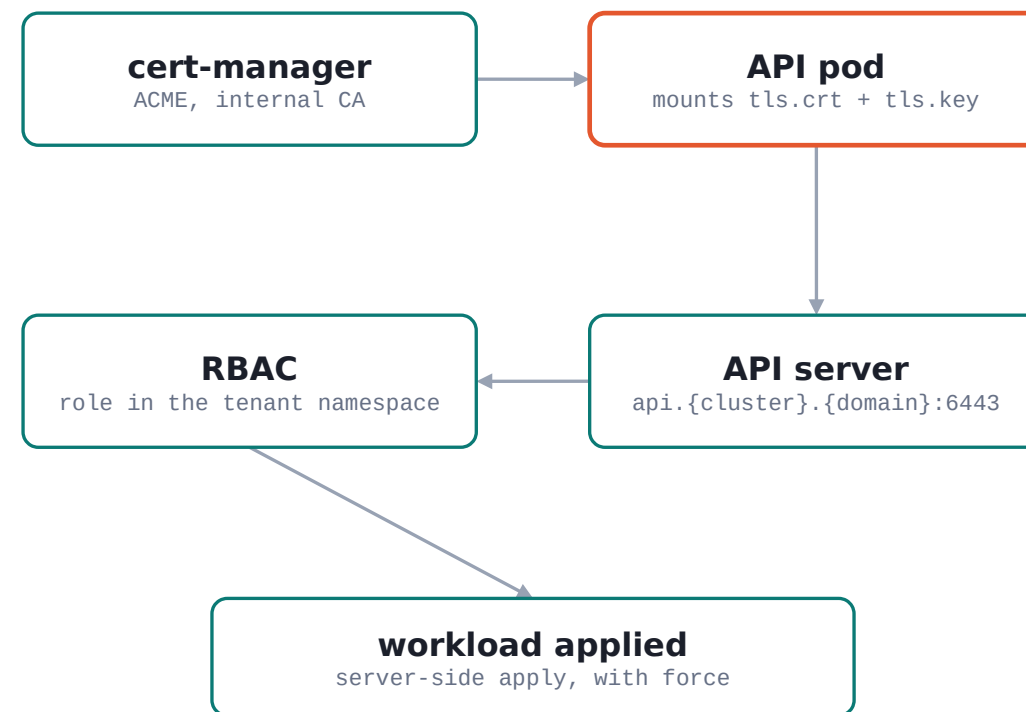
```
{
  "name": "billing", "port": 8080, "size": "small",
  "gitRepo": "https://git.internal/team-a/billing.git",
  "branch": "main", "runtime": "python", "version": "3.12",
  "regions": ["central", "south"],
  "env": [{ "name": "LOG_LEVEL", "value": "info" }]
}
```

202 Accepted

```
{
  "name": "billing", "status": "Pending",
  "hostname": "billing-team-a.serverless.example",
  "statusUrl": "/groups/team-a/functions/billing",
  "regions": [{ "region": "central", "status": "Pending" },
               { "region": "south", "status": "Pending" }]
}
```

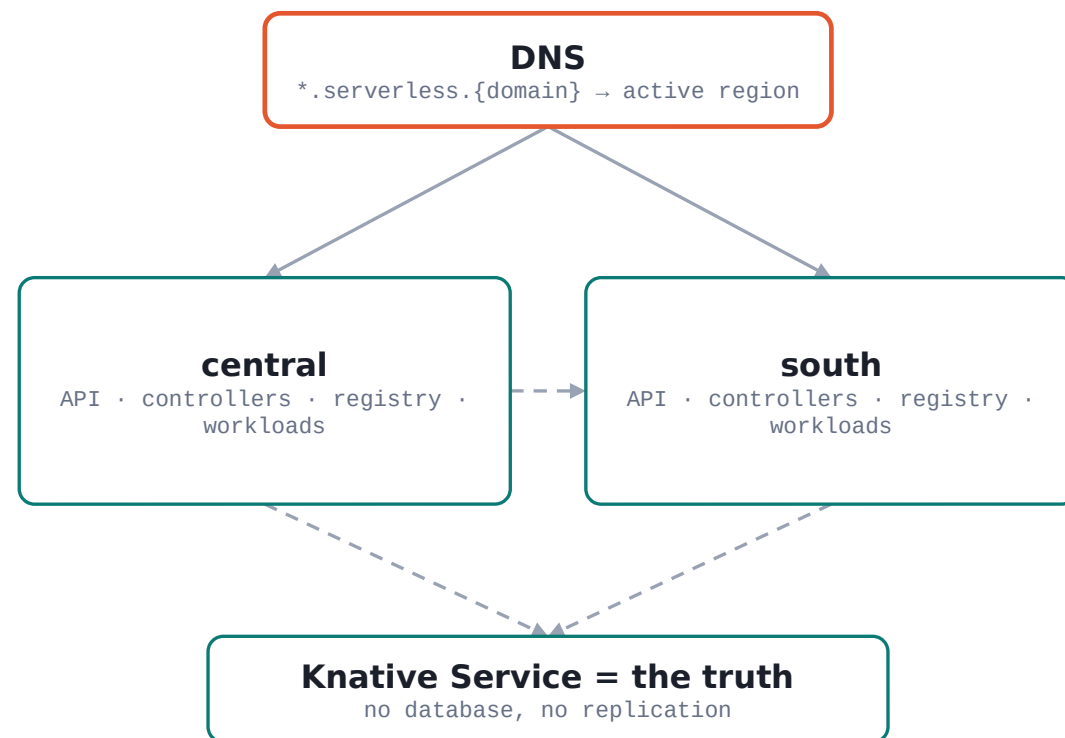
Talking to the cluster

- Client certificate mTLS, always
- No kubeconfig, no service-account token
- The certificate's CN is the user
- Address derived from the region name
- Every write is server-side apply



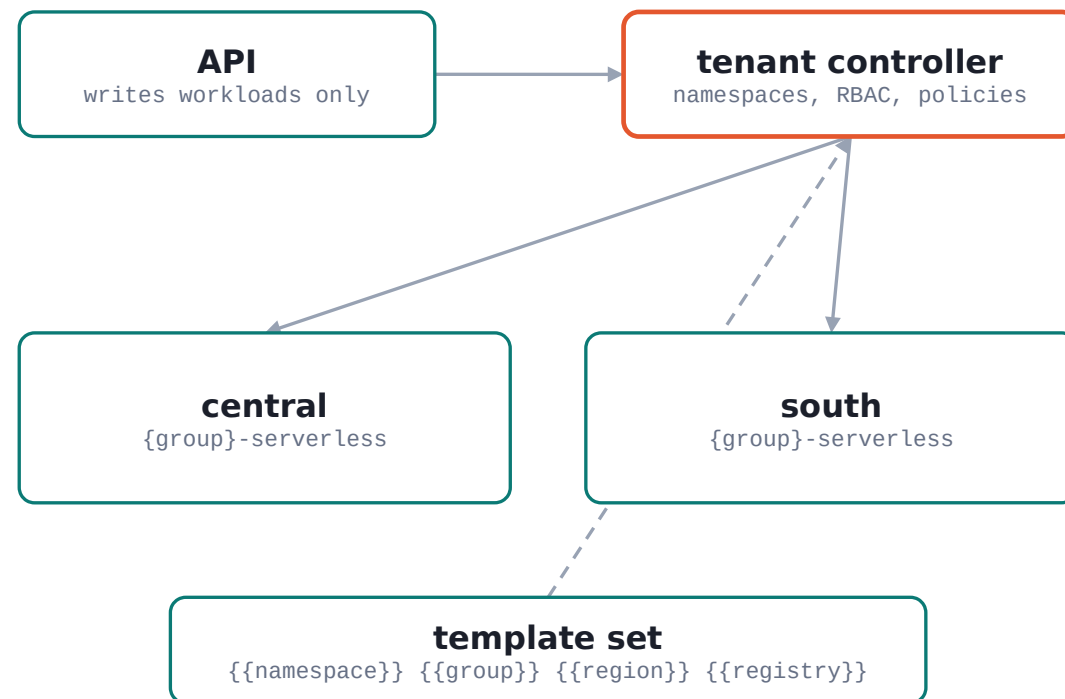
Active/active, two regions

- The API runs in both regions
- One deploy fans out to both
- No leader election, deterministic specs
- A region builds what it runs
- State lives in the Knative Service



The tenant controller

- A namespace per SSO group
- Separate process, separate certificate
- The API never creates namespaces
- Provision before every deploy, fail closed
- Reconcile the local cluster only
- Garbage-collect empty namespaces, opt-in

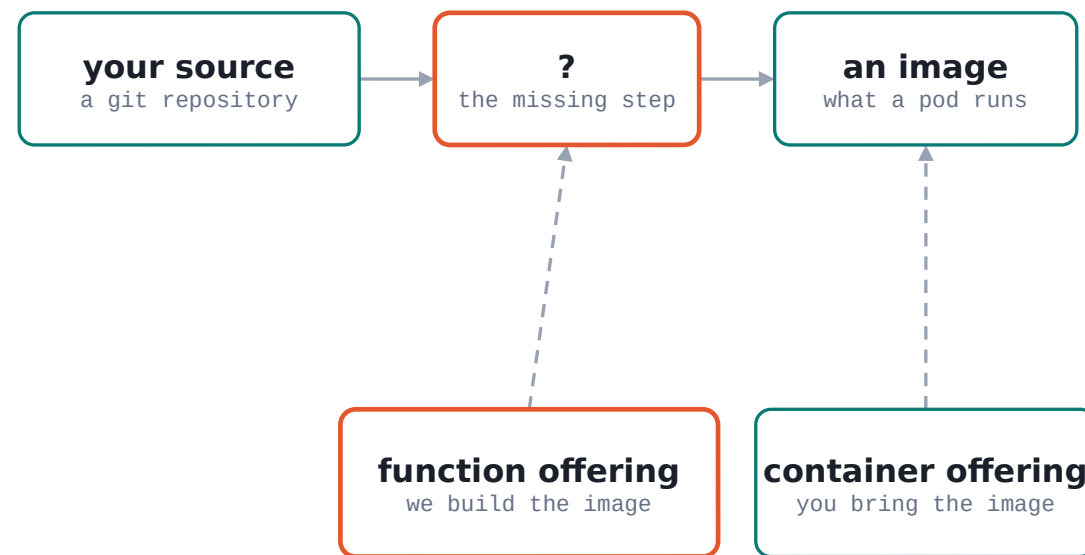


Functions & builds

'Here is my repo' hides a build.

Why a function needs a build

- A cluster cannot run a repository
- Kubernetes only ever starts images
- Someone must turn source into an image
- That step is the build
- For containers, the user already did it



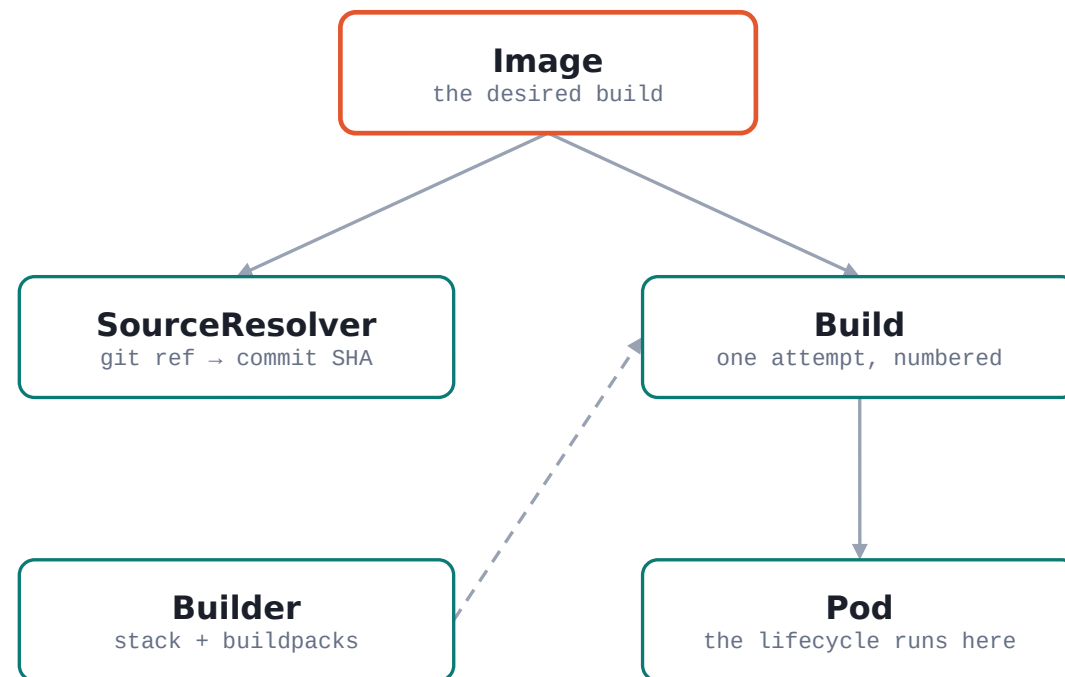
Buildpacks

- An open standard, not a Dockerfile
- Detection reads your source
- The same base image for everyone
- Rebase swaps that base, no rebuild
- Paketo is the family we mirror

	DOCKERFILE	BUILDPACKS
Who writes it	Every team, again	Nobody
Base image	Whatever was pinned	One curated stack
Language setup	Copied between repos	Detected from source
Patching the base	Rebuild and hope	Rebase, in seconds
Consistency	Per repository	Platform-wide

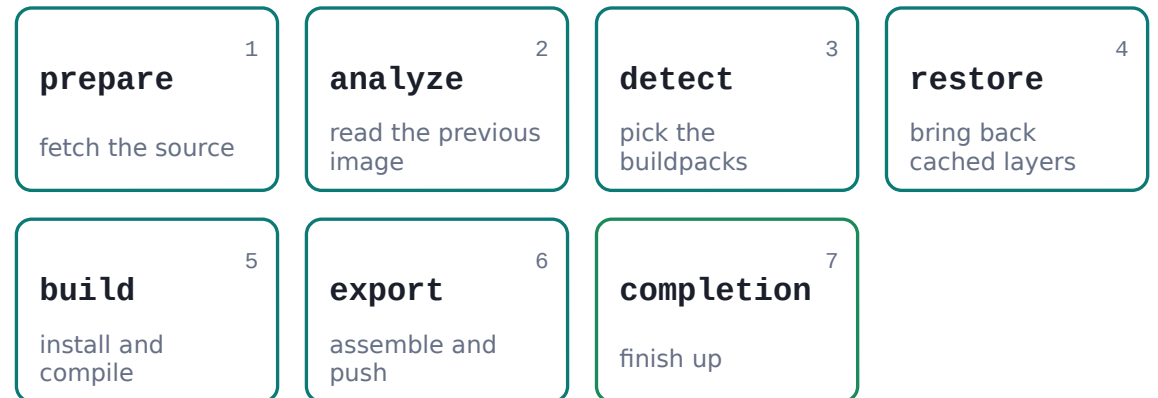
kpac: buildpacks on Kubernetes

- Buildpacks need something to run them
- kpac runs them as pods
- Image says what to build
- Build is a single attempt
- Builder holds stack and buildpacks



A build, phase by phase

- The lifecycle runs as init containers
- Each phase is a named container
- Each phase has its own log
- Cached layers make a rerun fast
- Export assemblies and pushes the image



Our kpack chart

- Not a fork, a Helm chart
- Packages upstream kpack faithfully
- CRDs as templates, for the webhook
- Creates the cluster build content
- Mirror scripts for the airgap

the chart

upstream kpack 0.18

CRDs

templated, not an untouched crds/

clusterBuild

ClusterStack + ClusterStore

credentials

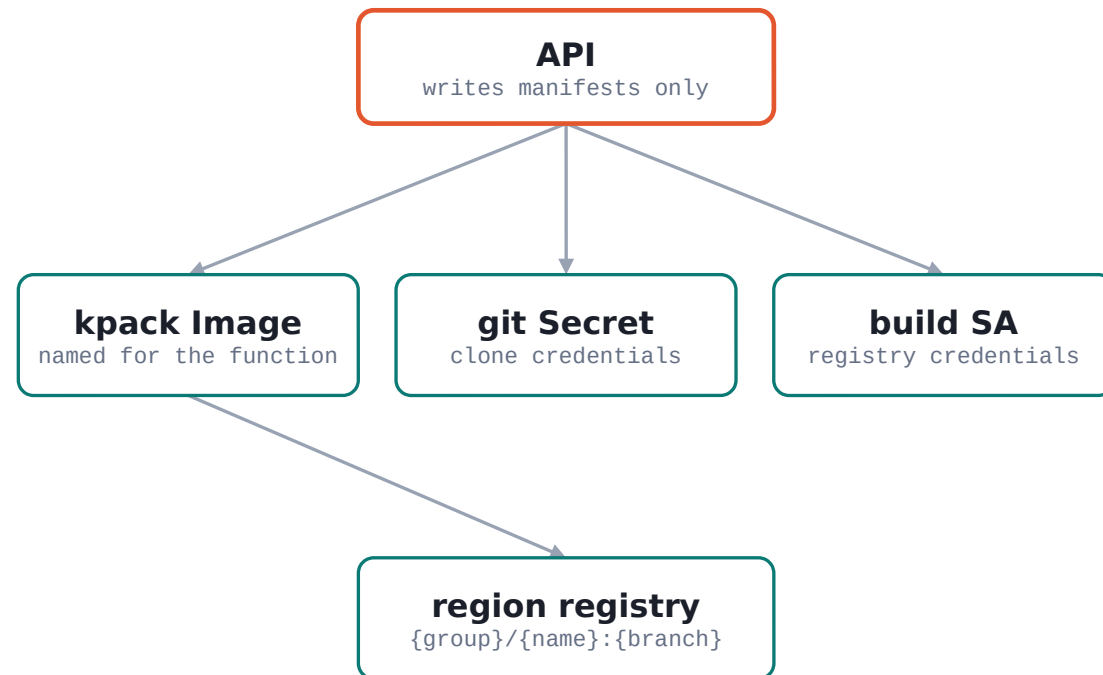
projected by external-secrets

scripts/mirror

images, buildpackages, runtimes

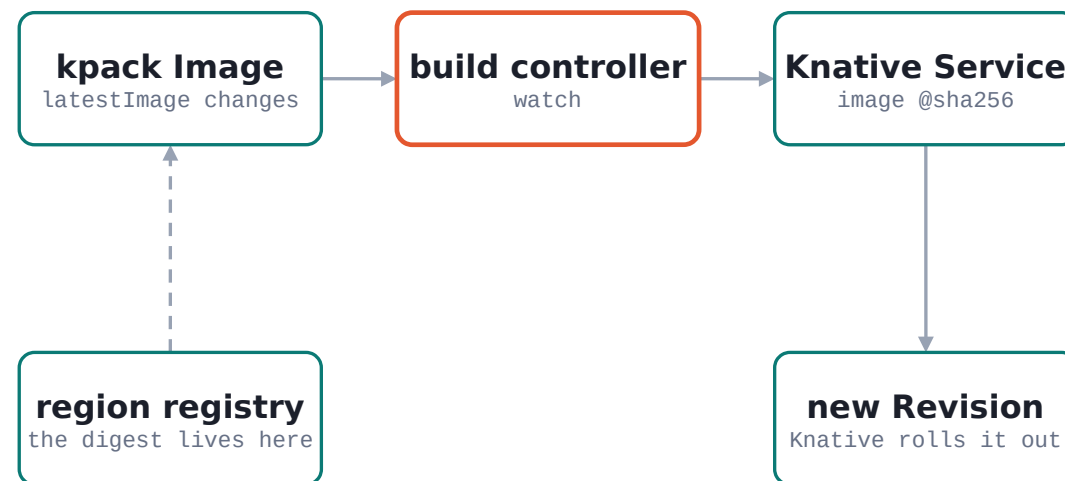
How the API uses kpack

- The API declares, it never builds
- One Image per function, per region
- A git Secret for the clone
- A build ServiceAccount for the registry
- Owned by the Knative Service



The build controller

- Watches Image.status.latestImage
- Applies the digest onto the Service
- Both ends stay in one region
- Only ever writes digests
- Ships with no web stack

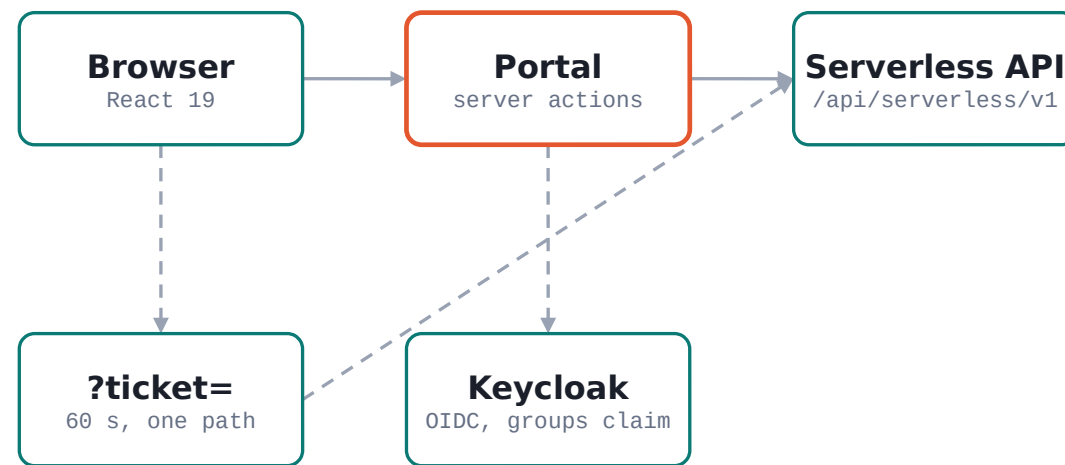


Portal & the future

A console, and a library for the next API.

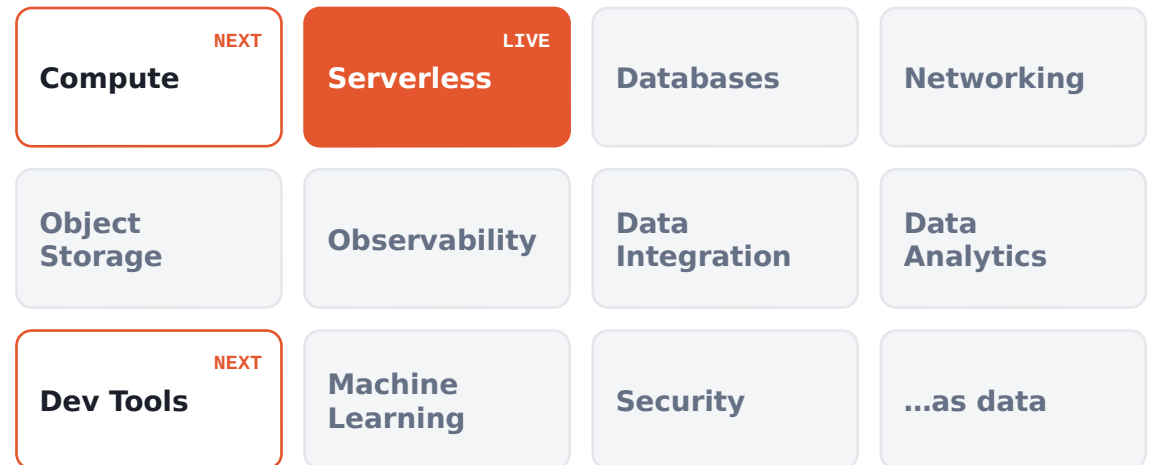
How the portal wires it

- Next.js 16, server components
- Keycloak token stays server-side
- The SSO group is the project
- Forms are built from /info
- Live logs over SSE tickets
- Airgap-native: no CDN, no fonts



A console, not a Serverless UI

- Serverless is live, Compute is next
- An offering is data, not code
- The shell is already generic
- Next: shared layer, generated client



cloudlet-apis: start at step five

- Extracted from the Serverless API
- core, web and auth extras
- Layering enforced by tests
- A new API wires in five steps
- Same group rules everywhere

[auth]

OIDC · admin key · token proxy ·
tickets

[web]

health · error envelope · offline docs

core

errors · names · logging · request id

Code in, a running service out.

4

repositories

3

services

23

endpoints

2

regions

3

runtimes

0

kubectl

- Operator owns Knative; we own the contract
- Deterministic specs, so no leader
- Builds declared, watched, rolled in
- The second product is cheaper

**Thank you.
Questions?**